Hierarchical Reinforcement Learning for Cryptocurrency Trading via Adaptive Strategy Selection

Ethan Farah
Departments of Computer Science and Mechanical Engineering
efarah@stanford.edu

Daniel Yang
Department of Computer Science
danielyang@stanford.edu

Abstract

We develop a hierarchical reinforcement learning framework for adaptive cryptocurrency trading at two timescales: macro-level (daily) and micro-level (sub-second). Rather than learning policies end-to-end, our approach couples domain-driven strategy families with Deep Q-Network agents that learn when *each hypothesis is reliable under non-stationary conditions. At the macro level, we trade Solana (SOL) spot using cross-asset momentum from SOL, BTC, ETH, and S&P 500. One DQN agent maps the input signals through its network to decide the sizing for buy/sell. At the micro level, we decompose perpetual futures trading into directional and volatility prediction tasks, defining 7 strategies each that adapt via online EWMA regression. Two DQN agents select strategies at matched frequencies (1s for direction, 30–60s for volatility), with forecasts translated into positions via fractional Kelly sizing. Both timescales show positive returns over the evaluation periods. On the macro level, our algorithm outperforms our baselines by 27%. On micro timescales, the agent achieves higher positive out-of-sample returns (final equity* $6.93\times$*, Sharpe* ≈ 0.98*). While transaction costs are not yet modeled, results suggest interpretable, regime-adaptive RL can outperform static baselines in crypto markets.*

1 Introduction

1.1 Problem Motivation

Cryptocurrency markets operate continuously and support an expanding ecosystem of spot and derivative trading opportunities. Among these, perpetual futures have become the dominant instrument for many high-frequency participants because they allow leveraged long and short exposure without requiring custody or borrowing of the underlying asset, while a periodic funding mechanism helps anchor contract prices to spot. This structure offers deep liquidity and a clean vehicle for rapid positioning, making perpetuals or "perps" a natural testbed for short-to-medium-horizon algorithmic trading.

At the same time, the design of robust trading algorithms in crypto remains challenging due to non-stationarity, heavy-tailed returns, and frequent regime changes across both macro and micro timescales. Fixed, single-strategy systems can perform well in extremely narrow conditions but often do terribly when volatility, cross-asset relationships, or order-flow dynamics shift. These properties motivate adaptive online strategy selection, where instead of committing to one predictive hypothesis, a learner can decide which hypothesis to trust right now based on realized performance and market features.

1.2 Problem Statement

We aim to develop reinforcement learning agents that trade SOL (both spot and perpetual futures) at two distinct timescales: macro (daily) and micro (sub-second)—by adaptively selecting among interpretable trading strategies that exploit cross-asset relationships with BTC and ETH. While supervised learning focuses on maximizing predictive accuracy, actual trading performance hinges on choosing the right strategy at the right time, with rewards revealed after an action.

We aim to trade both on the micro and macro level, which involve fundamentally different signals and constraints. Macro trading, which holds assets over multiple days, uses momentum on multi-day horizons and requires managing execution costs from larger position changes. Micro trading, which holds assets for several milliseconds to a few seconds, exploits millisecond lead-lag effects and order flow imbalances that dissipate within seconds. Since both tasks utilize different feature sets and exhibit distinct trading dynamics, we train separate RL agents for each timescale.

2 Background

2.1 Related Works

Our work exists within three main research problems: deep reinforcement learning for financial trading, reward engineering for risk-adjusted objectives, and online learning and adaptive strategy selection.

Mnih et al. (2015) (1) introduced Deep Q-Networks (DQNs), demonstrating that value-based RL with neural function approximation and experience replay could achieve human-level control in complex domains. We adopt the DQN architecture as our strategy selection mechanism at the high-frequency domain, using separate networks for spot and volatility prediction (1). More recently, Liu et al. (2021) (5) developed FinRL, a comprehensive framework for applying deep RL to finance, providing standardized environments and baselines for stock trading. While FinRL focuses on portfolio allocation across equities using daily data, our work extends RL trading to the cryptocurrency perpetual futures domain at two different resolutions and explicit cross-asset feature engineering (5).

Gort et al. (2022) (6) highlighted a critical challenge in applying deep RL to crypto trading: severe backtest overfitting when agents train on limited historical data. They found that many published crypto trading agents fail to generalize beyond their training periods due to overfitting to specific market regimes (6). To address this, we implement several overfitting mitigation strategies. First, we chunk episode resets that treat each data segment as independent. We also pre-define interpretable strategy families rather than learning truly from scratch. Third, we validate on multiple out-of-sample periods before final testing. Our approach is closer to strategy selection than strategy discovery, which reduces the hypothesis space and improves generalization (6).

A pioneering contribution by Moody and Saffell (1998) (4) introduced the differential Sharpe ratio (DSR) as an online reward signal for trading agents. Rather than optimizing raw returns, DSR provides a dense gradient signal that encourages the agent to increase risk-adjusted performance (the Sharpe ratio is defined as $\frac{\text{mean return}}{\text{return volatility}}$) (4). This is critical for trading, where high returns with extreme volatility are undesirable. We use Moody's DSR formulation with exponentially weighted moving averages (EWMA) to track running statistics, enabling our agents to directly optimize Sharpe ratio during training (4).

Most cryptocurrency trading research focuses on supervised prediction of price movements or portfolio optimization with daily rebalancing (5; 6). High-frequency crypto trading (sub-second) remains less explored in the academic literature, primarily due to data access barriers and the complexity of modeling microstructure effects. Our work bridges this gap by demonstrating that RL-driven adaptive strategy selection can improve performance at both daily and millisecond timescales. Unlike typical "black-box" deep RL approaches that learn policies end-to-end, our hierarchical design with interpretable strategy families makes the system more transparent and debuggable—critical properties for deployment in live trading where understanding why the agent makes decisions is essential for risk management.

2.2 Cross-asset Correlation

Historically, Bitcoin (BTC) and Ethereum (ETH) have been the most well known cryptocurrencies and also some of the largest by market capitalization. As of early December 2025, BTC's market cap is roughly $\$1.8$T, ETH's is roughly $\$0.4$T, and Solana's (SOL's) is roughly $\$0.08$T across major trackers (7; 8; 9). Market participants, specifically those with far more experience and money than Daniel and Ethan, watch BTC and ETH closely as "market leaders." When BTC and ETH move, these traders are often first to capitalize on any hidden signal. However, the effects of these trades should eventually propagate through the market to less liquid assets as time flows forward (a classic trading intuition). This creates a lead-lag relationship where price movement patterns in highly liquid assets lead to correlated motion in other crypto assets, like Solana (SOL). BTC and ETH are far more liquid cryptocurrencies than SOL, so market-moving information should appear in BTC/ETH prices first (which we were able to verify with a correlation analysis). SOL, being smaller, may react with a delay. This lead-lag relationship causes us to utilize their price movements as our primary input features for our SOL trading algorithm.

3 Macro Trading

3.1 Dataset

All crypto data was sourced from Binance vision, which serves as both a digital exchange and data warehouse for historical trades and prices. All data is held in zipped csv files, organized by timestamp. To construct our datasets, we created an automatic script to surgically download each csv of interest and extract our pricing information before concatenating the datapoints together.

For our first agent, which trades intra-day assets, we use 8-hour "kline" (candlestick) data sampled at 8-hour intervals, spanning September 14, 2020 to December 4, 2021. We take four timestamps per day $(T_{0h}, T_{8h}, T_{16h}, T_{24h})$. The input is a 16-dimensional feature vector of engineered momentum features derived from SOL, BTC, ETH, and the S&P 500, where S&P 500 daily closing prices over the same period are scraped from Yahoo Finance. Specifically, we include SOL price momentum at five horizons (8 hours, 16 hours, 24 hours, 2 days, 1 week), since this is the coin of interest. We garner extra trading signal from BTC, ETH, and S&P 500 price momentum at three horizons (24 hours, 2 days, 1 week). Lastly, we also concatenate the Solana trading volume plus its closing price. Momentum is defined as percent change over a given horizon

$$M_t = \frac{P_{24h} - P_{24h-t}}{P_{24h-t}}$$

Percentage normalizes for price scale. Because we take multi-day positions, the agent trades once per day at the T_{24h} close, outputting a discrete action in $\{\text{buy}, \text{hold}, \text{sell}\}$ that is executed as a spot position held over multiple days.

Since the stock market isn't open on weekends/holidays and only trades during market hours, we used linear interpolation to fill in the missing gaps for the S&P 500. For each time series with missing timestamps, values were imputed by dividing the interval between the two most recent observed S&P 500 trading prices into equally spaced points and assigning them in order to the missing time steps. Additionally, some of the data was missing or incomplete. To avoid training on fault/erroneous data, we dropped all datapoints with null values during preprocessing.

The dataset contains 1,899 daily observations spanning approximately 5.2 years, with 16 engineered features capturing cross-asset momentum and market dynamics. The state space includes these 16 features plus a normalized inventory position, resulting in a 17-dimensional state vector. We split the data chronologically with 90% (1,709 days) allocated for training and 10% (190 days) held out for evaluation.

3.2 Methodologies

One key difference on the macro level is we decided to trade the coin itself rather than futures because holding leveraged futures over longer horizons exposes us to unnecessary liquidation risk from ordinary volatility. We model the trading problem as a finite-horizon Markov Decision Process (MDP) with discrete time index $t \in \{0, \ldots, T-1\}$ corresponding to trading days. At each time step t, the agent observes a state $s_t \in \mathcal{S}$, learns to select the best action $a_t \in \mathcal{A}$, receives a scalar reward $r_t \in \mathbb{R}$, and transitions to the next state s_{t+1}.

The environment maintains the agent's inventory $I_t \in [0, 10]$, which represents how many coins are in our inventory. We cap the size of our inventory to prevent the size of our state space from outgrowing out dataset. We also introduce a cash balance constraint that starts at $\$10,000$. Let p_t denote the asset close price at time t, and let x_t denote the $F = 16$ raw market and cross-asset features extracted from a historical price dataset.

Table 1: Macro-level dataset statistics.

Property	Value
Total samples	1,899
Date range	2020-09-21 to 2025-12-02
Time span	≈ 5.2 years
Feature dimensions	
Raw features	16
State space (with inventory)	17
Data split	
Training set	1,709 days (90%)
Evaluation set	190 days (10%)
Action space	
Discrete actions	21 (inventory change: -10 to $+10$)

For each feature dimension $j \in \{1, \dots, F\}$ we compute the training-set mean μ_j and standard deviation σ_j and apply z-score normalization

$$\tilde{x}_t^{(j)} = \frac{x_t^{(j)} - \mu_j}{\sigma_j + 10^{-8}}$$

to improve numerical stability during training and ensure all features are on a similar scale, preventing features with larger magnitudes (such as price levels) from dominating the learning process. The small constant 10^{-8} in the denominator prevents division by zero when $\sigma_j = 0$. Let $\tilde{x}_t = (\tilde{x}_t^{(1)}, \dots, \tilde{x}_t^{(F)})$ denote the normalized feature vector, and let $I_{\max}$ be the maximum allowed inventory. We define our state vector as

$$s_t = \left[\, \frac{I_t}{I_{\max}},\ \tilde{x}_t \,\right] \in \mathbb{R}^{1+F}$$

Thus our state vector is 17-dimensional, with 16 coming from our dataset features and the last being our current inventory state. As mentioned before, our action space is defined as $\mathcal{A} = \{-I_{\max}, -I_{\max}+1, \dots, I_{\max}-1, I_{\max}\}$ where an action a_t represents the desired *change* in inventory relative to the current position. To enforce the cash constraint, the environment caps the trade such that the agent cannot buy more units than it can afford at the current price. Internally, the environment maps a_t to a target inventory level $I_t^{\text{desired}} = Min(Max(I_t + a_t, 0),\ I_{\max}))$. Finally, the inventory and cash are updated as

$$I_{t+1} = I_t + \Delta I_t,$$
$$C_{t+1} = C_t - \Delta I_t\, p_t.$$

3.3 Methods

The environment uses a price-change-based reward that explicitly ties the agent's performance to the profitability of its position after each action decision. Let p_t and p_{t+1} denote the close prices at time t and $t+1$, respectively, and let I_{t+1} be the inventory held immediately after executing the action at time t. The instantaneous reward is defined as

$$r_t^{\text{raw}} = (p_{t+1} - p_t)\, I_{t+1}.$$

This quantity is positive when the agent is net long and the price increases (or net short and the price decreases), and negative when the agent is positioned against the price movement. Large price swings or large position sizes can lead to very high-magnitude rewards, which can destabilize training.

To dampen these extremes while preserving the sign of the reward signal, we apply a sign-preserving logarithmic transform before using the reward in the Q-learning update:

$$r_t = \operatorname{sign}\!\big(r_t^{\text{raw}}\big)\ \log\!\big(1 + |r_t^{\text{raw}}|\big).$$

This transformation compresses large positive and negative values, making the learning problem numerically more stable while still encouraging the agent to seek higher raw profits.

The goal of the agent is to learn an *optimal action-value function* $Q^*(s,a)$, which for any state–action pair (s,a) gives the maximum achievable expected return when starting from s, taking action a, and acting optimally thereafter. Formally, we define Q^* as

$$Q^*(s,a) = \max_{\pi} \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \;\middle|\; s_t = s,\ a_t = a\right],$$

where π ranges over all possible policies, r_{t+k} are the (transformed) rewards, and $\gamma \in (0,1)$ is a discount factor that downweights distant future rewards and encourages policies that realize lower-variance returns sooner rather than later. In practice, we do not compute Q^* directly. Instead, we approximate it with a parametric function $Q_\theta(s,a)$ and update the parameters θ so that Q_θ satisfies the Bellman optimality relationship implied by the definition of Q^*.

Given a sampled transition $(s_t, a_t, r_t, s_{t+1}, d_t)$, where $d_t \in \{0,1\}$ indicates whether s_{t+1} is terminal, we form a temporal-difference (TD) target

$$y_t = r_t + \gamma(1 - d_t)\ \max_{a' \in \mathcal{A}} Q_{\theta^-}(s_{t+1}, a'),$$

using a separate target network Q_{θ^-} with periodically updated parameters. The network weights θ are then tuned by minimizing the mean squared error between the current Q-value estimate and this target:

$$\mathcal{L}(\theta) = \mathbb{E}\big[(y_t - Q_\theta(s_t, a_t))^2\big].$$

In this way, the definition of Q^* guides training via its associated Bellman equation, and the reward shaping in r_t directly influences the gradients used to update θ.

To represent Q_θ, we employ a relatively simple feedforward neural network (the Q-network). Given a state $s \in \mathbb{R}^{1+F}$, the network outputs a vector of Q-values, one for each discrete action in the action set $\mathcal{A}$. The architecture consists of two fully connected hidden layers with 128 units each and rectified linear layers as our activation layers:

$$h_1 = \mathrm{ReLU}(W_1 s + b_1),$$
$$h_2 = \mathrm{ReLU}(W_2 h_1 + b_2),$$
$$Q_\theta(s,\cdot) = W_3 h_2 + b_3,$$

where W_1, W_2, W_3 and b_1, b_2, b_3 are the learnable weight matrices and bias vectors. The final layer produces a $|\mathcal{A}|$-dimensional output, and the entry corresponding to action a is interpreted as $Q_\theta(s,a)$. During training, these parameters are iteratively adjusted using stochastic gradient descent (Adam) to reduce the loss in Eq. (15), thereby driving Q_θ toward the optimal action-value function Q^*.

We also use an experience replay buffer $\mathcal{D}$ to reduce temporal correlations and improve sample efficiency. Each step stores $(s_t, a_t, r_t, s_{t+1}, d_t)$, where d_t is a terminal flag, and training samples i.i.d. mini-batches uniformly from $\mathcal{D}$. Additionally, we train with an ϵ-greedy policy and exponential decay.

$$\epsilon(t) = \epsilon_{\text{end}} + (\epsilon_{\text{start}} - \epsilon_{\text{end}}) \exp(-t/\tau).$$

And at each step, the agent selects a random action with probability $\epsilon(t)$ to explore all potential states, otherwise $a_t = \arg\max_{a \in \mathcal{A}} Q_\theta(s_t, a)$ and goes down the believed most optimal path.

The agent interacts with the environment until the end of the price series is reached. For $t = 0, 1, \dots$ until terminal step:

1. Select action a_t using the ϵ-greedy policy.
2. Execute a_t in the environment to obtain (s_{t+1}, r_t, d_t) subject to our constraints
3. Store $(s_t, a_t, r_t, s_{t+1}, d_t)$ in the replay buffer $\mathcal{D}$.
4. Once $|\mathcal{D}| \geq B$, sample a mini-batch and perform a gradient step on $\mathcal{L}(\theta)$.
5. Every fixed number of gradient steps, update the target network parameters.

3.3.1 Hyperparameter Selection

Training on the first 90% of samples, which comprise the training set, the agent was optimized using Adam with learning rate $\alpha = 10^{-3}$, discount factor $\gamma = 0.99$, and a target network update frequency of 500 steps. And experience replay was performed with a buffer capacity of 10,000 transitions and minibatch size of 64. Exploration followed an ϵ-greedy schedule with $\epsilon_{\text{start}} = 1.0$, $\epsilon_{\text{end}} = 0.05$, and exponential decay over 5,000 steps. The environment was initialized with \$10,000 cash and a maximum inventory constraint of 10 SOL units.

3.3.2 Performance Metrics

Table 2 summarizes the evaluation results on the held-out test period. The RL agent achieved a final portfolio value of \$10,611.17, representing a positive P&L of \$611.17 (+6.11%) over the 190-day evaluation window. Notably, this performance occurred during a bear market period where SOL price declined from \$176.64 to \$138.65 (a -21.5% price drop), demonstrating the agent's ability to adapt to adverse market conditions.

Figure 1: Macro training loss over training steps

The agent's action distribution reveals a predominantly defensive strategy: negative actions (selling/reducing inventory) accounted for 60% of all decisions, with the most frequent action being -10 (maximum sell) occurring 23 times. This behavior is consistent with the bearish market regime, suggesting the agent learned to reduce exposure during downtrends rather than maintaining a static long position.

3.3.3 Baseline Comparisons

We compared the RL agent against two baselines: (1) a *buy-and-hold* strategy that purchases the maximum number of SOL possible at the start price and holds until the end, and (2) a *bogo trader* that selects random actions uniformly from the action space at each time step.

The buy-and-hold baseline suffered a loss of \$2,150.70 (-21.51%), directly reflecting the price decline over the evaluation period. The bogo trader achieved a loss of \$184.61 (-1.85%), performing better than buy-and-hold due to its randomness but still underperforming the RL agent.

The RL agent outperformed buy-and-hold by \$2,761.87 (+27.62% relative improvement) and outperformed the bogo trader by \$795.78 (+7.96% relative improvement). From these results, we can deduce that our learned policy successfully identified corrlated market conditions which outperforms the market.

3.3.4 Interpretations

The agent's strong performance during a bear market suggests it learned to recognize adverse conditions and adjust position sizing accordingly. The heavy skew toward selling actions indicates the policy learned to reduce exposure when market conditions deteriorate, rather than maintaining a fixed long bias. The 7.96% outperformance over random action selection (bogo trader) validates that the learned Q-function does indeed captures meaningful market structure. The agent is not merely benefiting from favorable market conditions, but actively making better decisions than random chance would suggest.

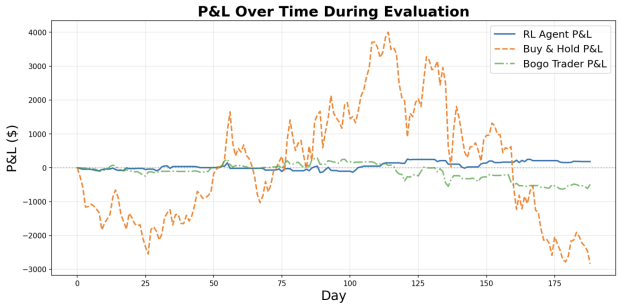

Figure 2: P&L of Algorithm Over Time

Table 2: Macro agent evaluation on held-out test set (190 days).

Strategy	Final Value	Return (%)
RL Agent	\$10,611	+6.11
Buy & Hold	\$7,849	−21.51
Bogo Trader	\$9,815	−1.85
RL vs. B&H: +27.6%, vs. Bogo: +8.0%		

While the results are encouraging, several limitations should be noted from our study. Firstly, the evaluation period only represents a single market regime (bear market), and performance may vary across different market conditions. We also ran the algorithm over a bull market, which it also slightly outperformed the market. Secondly, transaction costs, slippage, and market impact are not modeled in the current evaluation. Future work could include cost-aware rewards, longer evaluation periods spanning multiple market cycles, and stress testing under various volatility regimes, but such length amount of data might be a challenge to obtain given Solana's lifespan and recency.

4 High Frequency Trading

4.1 Dataset

Our micro-level dataset is built from aggregated trades of Binance USDT-settled perpetual futures for BTC, ETH, and SOL from January 1, 2025 to April 10, 2025. We build a 420-dimensional state vector consisting of log returns, momentum (computed over 7 windows ranging from 100 ms to 60 minutes), sampled at 100-millisecond intervals. The input to this system is a 420-dimensional state vector consisting of log returns, momentum (computed over 7 windows ranging from 100 ms to 60 minutes), order flow imbalance, and realized volatility features across all three assets.

Each asset's trade stream is discretized to a shared 100ms master index. Within each 100ms bucket, we use the last observed trade price as the bucket price, sum total quantity as traded volume, and compute signed volume using `is_buyer_maker` as a proxy for taker-side direction. We then align BTC, ETH, and SOL to the same timeline, forward-fill prices across empty buckets, and set missing quantity and signed volume to zero. This yields synchronized cross-asset time series at sub-second resolution while preserving the short-horizon flow and return dynamics needed for our lead-lag and regime-selection hypotheses.

4.1.1 Forward Targets

Our strategy families are trained to forecast short-horizon SOL direction and volatility, matching the decision cadence of our hierarchical agent. The spot forecasting target is the next 100ms SOL return, $y_t^{\text{return}} = r_{t+1}^{\text{SOL}}$, which supports second-level strategy selection that can exploit transient microstructure signals without relying on a single static predictor.

For volatility-aware sizing, we also construct forward realized volatility labels over 30s and 60s horizons, defined as $y_t^{\text{vol}_{30s}} = \sigma_{t\to t+300}^{\text{SOL}}$ and $y_t^{\text{vol}_{1m}} = \sigma_{t\to t+600}^{\text{SOL}}$ respectively, and computed from forward returns on the 100ms grid. These targets allow the volatility agent to choose among competing hypotheses (persistence, mean reversion, and cross-asset volatility transmission) and provide the uncertainty estimates required by our calibrated Kelly-style position-sizing rule.

4.1.2 Feature Screening

To choose features without overfitting our entire pipeline to the final evaluation window, we first ran a large-scale lead-lag correlation scan on a fixed chunk of the micro-level data that would importantly *not* be used for our train, val, and test loops. Concretely, using the aforementioned 100ms grid, we first constructed lagged versions of our candidate microstructure features (defined in detail in the Feature Engineering subsection) across multiple horizons, and computed Pearson correlations against forward SOL return and volatility targets at matched horizons. Because this analysis involves tens of millions of observations per feature-target pair, statistical significance is not a meaningful filter; instead, we focus on effect sizes and correlations as a descriptive prior for which both intra- and cross-asset signals appear strongest at short timescales. We then used these results to decide on a compact, interpretable strategy library rather than expanding the state space indiscriminately.

Table 3 confirms the lead-lag prior from Section 2.2: ETH and BTC log returns show correlations of approximately 0.12 with 100ms-ahead SOL returns, while SOL's own features are weaker (around 0.03).

For volatility, SOL realized volatility at 30s and 5m scales correlates at roughly 0.52 with 30s and 1m forward targets, supporting persistence-based volatility strategies.

Table 3: Selected top correlations from the feature screening analysis (targets are SOL).

Feature	Target	Corr.
`eth_log_return_cur`	`fwd_return_100ms`	0.120
`btc_log_return_cur`	`fwd_return_100ms`	0.114
`eth_momentum_100ms_cur`	`fwd_return_100ms`	0.111
`eth_flow_imbalance_1s_cur`	`fwd_return_1s`	0.070
`sol_vw_flow_imbalance_1s_cur`	`fwd_return_1s`	0.038
`sol_realized_vol_30s_cur`	`fwd_vol_30s`	0.525
`sol_realized_vol_5m_cur`	`fwd_vol_1m`	0.521

4.1.3 Feature Engineering

Our state representation is built from a compact set of interpretable microstructure features computed independently for each asset $a \in \{\text{BTC}, \text{ETH}, \text{SOL}\}$. The full 420-dimensional feature vector results from 35 base features per asset, computed at four short lags (current, 100ms, 500ms, 1s) across three assets, giving $35 \times 4 \times 3 = 420$. See the prior section and Appendix A for more information on why we picked these features.

We define 100ms log returns as $r_t^{(a)} = \log\left(\frac{p_t^{(a)}}{p_{t-1}^{(a)}}\right)$ where $p_t^{(a)}$ denotes the last-trade price for asset a in bucket t. Log returns are additive over time and provide a stable scale for comparing short-horizon movement across assets.

To capture short-to-medium trend strength in a way that remains stable under rapid non-stationarity, we compute an exponentially weighted moving average (EWMA) baseline of price at multiple halflives h, $\text{EWMA}_{t,h}^{(a)} = \text{EWM}\left(p_t^{(a)}; h\right)$[1] and define momentum as the normalized deviation from this smoothed reference, $\text{mom}_{t,h}^{(a)} = \frac{p_t^{(a)} - \text{EWMA}_{t,h}^{(a)}}{\text{EWMA}_{t,h}^{(a)}}$.

We include halflives spanning 100ms, 1s, 5s, 30s, 5m, 15m, and 60m to represent both immediate microstructure effects and slower intraday drift.

We measure directional pressure using a bucket-level direction proxy derived from signed volume (volume of trade multiplied by $+1$ for buy orders and -1 for sell orders). Let $\text{sv}_t^{(a)}$ and $\text{q}_t^{(a)}$ denote signed volume and total quantity for asset a in bucket t, and define $d_t^{(a)} = \text{sign}\left(\text{sv}_t^{(a)}\right)$. Our baseline flow-imbalance feature is the EWMA-smoothed direction, $\text{FI}_{t,h}^{(a)} = \text{EWM}\left(d_t^{(a)}; h\right)$ and our volume-weighted variant is $\text{VWFI}_{t,h}^{(a)} = \frac{\text{EWM}\left(\text{sv}_t^{(a)}; h\right)}{\text{EWM}\left(\text{q}_t^{(a)}; h\right) + \epsilon}$, where ϵ is a small constant for numerical stability. These two families allow the agent to distinguish between persistent directional flow and direction that is amplified by abnormal trading intensity.

We represent uncertainty through realized volatility at multiple timescales using an EWMA estimate of squared 100ms returns $\text{rv}_{t,h}^{(a)} = \sqrt{\text{EWM}\left(\left(r_t^{(a)}\right)^2; h\right)}$ computed at halflives 100ms, 1s, 30s, 5m, 15m, and 60m. This captures both bursty micro volatility and longer-lived regime shifts that matter for risk-adjusted position sizing.

To quantify volume "surprise" relative to a recent baseline, we compute $\text{VI}_{t,h}^{(a)} = \frac{\text{q}_t^{(a)}}{\text{EWM}\left(\text{q}_t^{(a)}; h\right) + \epsilon}$ using the same family of halflives as momentum and flow. This feature helps separate low-activity drift from high-intensity bursts that often coincide with short-horizon directional moves.

To explicitly model very recent cross-asset delay structure, we compute each base feature at lags $\{0, 100\text{ms}, 500\text{ms}, 1\text{s}\}$, enabling the agent to compare, for example, BTC flow or ETH momentum from one second ago against current SOL dynamics.

4.2 Methods

4.2.1 System Architecture

Our system employs a two-layer hierarchical architecture with a fixed bet-sizer layer. The strategy layer consists of 7 spot strategies and 7 volatility strategies, each implementing online EWMA regression to adapt to market conditions. We utilize two independent Deep Q-Network agents—one for spot strategy selection and one for volatility strategy selection—that learn which strategy to deploy at each time step. Finally, the position sizing layer applies a modified Kelly criterion to combine strategy predictions and volatility estimates into concrete position sizes to execute.

The system uses hierarchical decision frequencies matched to the natural timescales of different market phenomena. The spot agent selects directional strategies every 1 second (10 rows of 100ms data), as directional signals are ephemeral and require rapid response. To avoid rundundant calculations, the volatility agent updates every 30 seconds (300 rows) or every 60 seconds (600 rows), recognizing that volatility regimes evolve slowly over minutes rather than seconds.

4.2.2 Strategy Families with Online EWMA Regression

Rather than learning a trading policy from scratch in a high-dimensional continuous action space, we define interpretable strategies that implement specific market hypotheses. The reinforcement learning agent learns when each hypothesis applies rather than discovering price dynamics de novo. This approach combines domain knowledge with data-driven learning, making the system more sample-efficient and interpretable.

Each strategy maintains an online linear model, $\hat{y}_t = \mathbf{w}^\top \mathbf{x}_t$, that adapts continuously to non-stationarity. Given a halflife $\tau = 600$ seconds, the decay rate is $\lambda = 2^{-1/\tau}$. At each timestep, we update exponentially weighted covariance matrices, $\mathbf{\Sigma}_{XX}^{(t)} = \lambda \mathbf{\Sigma}_{XX}^{(t-1)} + (1-\lambda)\mathbf{x}_t\mathbf{x}_t^\top$, $\mathbf{\Sigma}_{Xy}^{(t)} = \lambda \mathbf{\Sigma}_{Xy}^{(t-1)} + (1-\lambda)\mathbf{x}_t y_t$, and solve for weights

[1] Here EWM denotes an exponentially weighted moving average, which assigns exponentially decaying weights to past prices so that more recent observations matter more. The halflife h is the timescale at which the weight on an observation decays to one half of its previous value, providing a convenient, interpretable way to control how quickly the baseline adapts to new information.

with ridge regularization, $\mathbf{w} = (\mathbf{\Sigma}_{XX} + \alpha \mathbf{I})^{-1} \mathbf{\Sigma}_{Xy}, \quad \alpha = 10^{-6}$. The ridge term is essential for numerical stability but small enough to minimally bias estimates.

For strategies with directional hypotheses, we enforce the hypothesis via a direction parameter. For strategy s with direction $d_s \in \{-1, 0, +1\}$, the prediction is $\hat{y}_s = d_s \cdot \left|\mathbf{w}_s^\top \mathbf{x}\right|$. When $d = +1$ (momentum), we predict in the same direction as the raw signal. When $d = -1$ (mean reversion), we predict in the opposite direction. When $d = 0$ (flat), we predict zero. This structure enforces the strategy hypothesis while allowing the regression to learn optimal magnitudes from data.

We define seven *spot strategies*, each testing a different hypothesis about directional price movements with relevant features determined from the prior lead-lag screening. The `flat` strategy predicts zero return and takes no position, allowing the agent to abstain when the market is unreadable. The `btc_leads` strategy uses Bitcoin returns, momentum, and flow imbalance features to test whether BTC leads SOL, while `eth_leads` uses analogous Ethereum features. The `both_lead` strategy combines features from both BTC and ETH, capturing scenarios where multiple assets provide complementary signals. For SOL-specific dynamics, `sol_momentum` tests whether SOL trends persist (with direction parameter $d = +1$), while `sol_mean_revert` tests the opposite hypothesis that SOL overreacts and reverts (with $d = -1$). Finally, `flow_driven` aggregates order flow imbalance across all three assets, testing whether net buying/selling pressure predicts subsequent returns.

This design explicitly separates BTC and ETH signals because sometimes only one asset leads. Including both momentum and mean-reversion strategies with opposite direction parameters allows the agent to learn which regime applies. The flat strategy prevents the agent from being forced to take a position when uncertainty is high, reducing unnecessary losses.

We also define seven *volatility strategies* to predict future realized volatility. The `flat` strategy provides a constant baseline estimate. The `vol_30s_persists` strategy predicts 30-second forward volatility using recent 30-second realized volatility, testing volatility clustering. Conversely, `vol_30s_mean_reverts` predicts that volatility spikes are temporary and will revert. We include analogous strategies for 1-minute horizons (`vol_1m_persists` and `vol_1m_mean_reverts`). The `vol_cross_asset` strategy uses BTC and ETH volatility to predict SOL volatility, capturing cross-market contagion effects. Finally, `vol_from_flow` tests whether flow imbalance spikes predict volatility increases.

These volatility forecasts provide the uncertainty estimates required by our Kelly-style position sizing rule and allow exposure to adapt to changing regimes.

4.2.3 Deep Q-Network Agents

We employ standard Deep Q-Networks (DQN) with separate networks for spot and volatility agents. The state space consists of market features only: $\mathbf{s}_t \in \mathbb{R}^{420}$, comprising lagged returns, momentum indicators, flow imbalance, and realized volatility across BTC, ETH, and SOL. Critically, the current position is not observed by the agent—strategies are purely predictive, not reactive to portfolio state.

The action spaces are discrete and compact. The spot agent selects among 7 directional strategies, while the vol agent independently selects among 7 volatility strategies. This hierarchical factorization reduces the joint action space from $7 \times 7 = 49$ to $7 + 7 = 14$ learnable mappings, improving sample efficiency.

Each Q-network is a two-layer feedforward network with ReLU activations: Input$(420) \to$ Linear$(128) \to$ ReLU $\to$ Linear$(128) \to$ ReLU $\to$ Linear(7). This architecture suffices because the strategies themselves handle feature-to-prediction mapping; the DQN only needs to learn strategy selection.

We use a target network for stability with hard updates every 1,000 steps. The loss is the standard temporal difference error, $\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a)\right)^2\right]$, with $\gamma = 0.99$ and replay buffer capacity 100,000. Exploration uses ϵ-greedy with exponential decay from 1.0 to 0.01 over a halflife of 50,000 steps.

4.2.4 Differential Sharpe Ratio Reward

Our objective is to maximize risk-adjusted performance. The Differential Sharpe Ratio (DSR) provides a dense per-timestep reward that approximates the marginal contribution of each return to the overall Sharpe ratio. We maintain running estimates of mean return A_t and mean squared return B_t using exponential weighting with learning rate $\eta = 0.01$: $A_t = A_{t-1} + \eta(r_t - A_{t-1})$ and $B_t = B_{t-1} + \eta(r_t^2 - B_{t-1})$. The DSR gradient reward is $D_t = \frac{B_{t-1}\Delta A_t - \frac{1}{2}A_{t-1}\Delta B_t}{\left(B_{t-1} - A_{t-1}^2\right)^{3/2} + \epsilon}$, where $\Delta A_t = r_t - A_{t-1}$, $\Delta B_t = r_t^2 - B_{t-1}$, and $\epsilon = 10^{-8}$ prevents division by zero. A positive D_t indicates that r_t improved Sharpe by increasing the running mean or reducing effective variance; a negative D_t indicates the opposite. This reward discourages high-variance strategies with low mean and aligns the RL objective with our evaluation metric.

4.2.5 Kelly Position Sizing with Time-Scale Normalization

Our features and targets span heterogeneous time horizons. To avoid mixing incompatible units within the position-sizing layer, we normalize return and volatility predictions to a common daily scale before applying Kelly. Returns scale linearly with time, while volatility scales with the square root of time. For spot strategies predicting 100ms returns, we scale $\hat{\mu}_{\text{daily}} = \hat{y}_{\text{spot}} \times 864{,}000$. For volatility predictions, we use $\hat{\sigma}_{\text{daily}} = \begin{cases} \hat{y}_{\text{vol}} \times \sqrt{2880}, & \text{30s horizon}, \\ \hat{y}_{\text{vol}} \times \sqrt{1440}, & \text{1m horizon}. \end{cases}$ Given daily return prediction $\hat{\mu}_{\text{daily}}$ and daily volatility prediction $\hat{\sigma}_{\text{daily}}$, the raw Kelly fraction is $f_{\text{raw}} = \frac{\hat{\mu}_{\text{daily}}}{\hat{\sigma}_{\text{daily}}^2}$. We blend predicted and realized volatility to reduce sensitivity to estimation error, $\sigma_{\text{mixed}}^2 = \alpha \cdot \sigma_{\text{predicted}}^2 + (1 - \alpha) \cdot \sigma_{\text{realized}}^2, \quad \alpha = 0.5$, and apply fractional Kelly scaling, $f_{\text{scaled}} = 0.25 \cdot f_{\text{raw}}$. Finally, we enforce leverage limits: $f = \text{clip}(f_{\text{scaled}}, -1.0, 1.0)$.

4.2.6 Data Processing and Training Procedure

Time series data exhibit temporal dependency, making random train/test splits invalid. We use a strict chronological split (consistent with the dataset configuration reported in Section 3.1): the first 70% of data for training, the next 15% for validation, and the final 15% for testing.

To prevent subtle forms of leakage for online components, we implement careful state management. Each epoch begins by resetting the environment and restoring EWMA strategy weights to their post-warmup state. During training, EWMA strategies adapt online and DQN agents learn. Before validation and testing, we save EWMA states and DQN weights, run evaluation with learning disabled, then restore both to their pre-evaluation values. This ensures that validation and test performance does not influence subsequent training.

We train for 6 epochs and select the model that achieves the highest validation Sharpe ratio. The final test evaluation uses the best validation checkpoint, ensuring the reported results reflect out-of-sample generalization.

4.3 Results

4.3.1 Hyperparameters and Training Configuration

Hyperparameters were selected via full factorial grid search over 5 parameters, yielding 243 configurations evaluated on validation Sharpe ratio. The search varied learning rate $\alpha \in \{10^{-5}, 10^{-4}, 10^{-3}\}$, hidden layer size $\in \{64, 128, 256\}$, EWMA regression halflife $\in \{300, 600, 1200\}$ seconds, fractional Kelly multiplier $\in \{0.10, 0.25, 0.50\}$, and discount factor $\gamma \in \{0.95, 0.99, 0.995\}$. The winning configuration ($\alpha = 10^{-4}$, hidden $= 128$, halflife $= 600$s, Kelly $= 0.25$, $\gamma = 0.99$) achieved the highest validation Sharpe. The test set was held out entirely during hyperparameter selection to prevent selection bias.

We trained two hierarchical DQN agents with a shared, lightweight MLP architecture (128-unit hidden layers with ReLU activations) and independent output heads for spot and volatility strategy selection. The spot agent acts at a 1-second cadence, while the volatility agent acts at a 30-second cadence (see Section 4.1.1). Both agents were optimized with Adam ($\alpha = 10^{-4}$), discount factor $\gamma = 0.99$, a target-network update period of 1,000 steps, and an experience replay buffer of size 100,000 with minibatch sampling.

Exploration followed an ϵ-greedy schedule with decay to a small floor, ensuring broad early regime coverage and stable late-stage exploitation. Online EWMA regressions used the halflife and ridge settings described in Section **??**, and all return/volatility forecasts were normalized to a common daily scale prior to position sizing using the daily-ization rules defined in Section 4.2.4. Position sizing followed Section 4.1.5.

4.3.2 Training Dynamics and Generalization

Figure 3 summarizes training behavior. We observe a pronounced divergence between training and validation equity early in learning: training equity can explode to implausible levels, while validation remains comparatively stable. In this setting, this pattern is consistent with the combination of (i) strong function approximation, (ii) online adaptive strategy learners, and (iii) a convex position-sizing rule that can magnify small in-sample edges. Put differently, the agent is not merely fitting a static predictor; it is learning a regime-switching controller on top of online regressors, and the resulting in-sample pathwise fit can be unrealistically strong.

Crucially, the validation curve does *not* track these training spikes. This suggests that the chronological holdout is acting as a meaningful out-of-sample filter, and that extended training does not monotonically improve generalization in this low signal-to-noise domain. Practically, the best checkpoint is therefore determined by validation performance rather than by late-epoch training equity.

From a market-microstructure perspective, this behavior is expected: sub-second crypto returns are extremely noisy, so a flexible controller can easily over-specialize to the realized training path. From an RL perspective, the hierarchical factorization helps mitigate action-churn, but Q-learning can still over-amplify transient edges when the reward is risk-adjusted yet still sensitive to compounding through position sizing. This explains why the most reliable out-of-sample performance appears in earlier checkpoints.

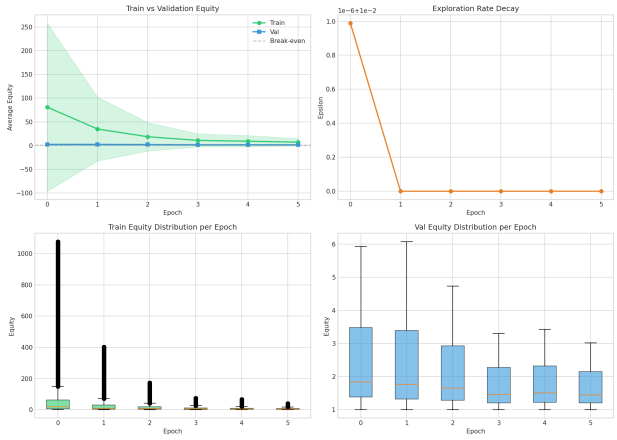

Figure 3: Top-left: Train vs. validation average equity across epochs with dispersion, illustrating strong in-sample amplification in early epochs and markedly more stable out-of-sample behavior as time evolves. Top-right: ϵ-greedy exploration decay schedule used by the DQN agents. Bottom row: Distributional summaries of equity by epoch for train and validation, highlighting the extent to which early training performance can be dominated by pathwise over-specialization and compounding effects under aggressive position sizing.

4.3.3 Test Set Performance

Table 4: Test set performance metrics for the selected checkpoint.

Metric	Value	Metric	Value
Mean equity	3.06×	Mean step Δ	$+6.9 \times 10^{-5}$
Median equity	1.79×	Std step Δ	3.8×10^{-4}
Std equity	2.11	Max gain	+0.0194
Final equity	6.93×	Max loss	−0.0115
Min equity	1.00×	Sharpe	0.98
Max equity	6.93×	Final PnL	+5.93

We evaluated the best validation checkpoint on the held-out test window. Table 4 and Figure 4 summarize the resulting equity and step-level return statistics. The agent achieves a strong positive cumulative PnL over the test period with modest per-step edge, implying that performance is driven by consistent small advantages compounded over many decisions rather than by a single dominant trade.

The distribution of equity step changes is tightly concentrated near zero with mild positive skew. This shape is characteristic of microstructure-scale trading under conservative risk limits: most decisions are close to flat in realized impact, while a smaller fraction of favorable regime selections drive the long-run drift in equity. In our framework, this is exactly the desired behavior of the `flat` option and the cross-asset lead strategies: abstain when the short-horizon state is ambiguous and scale exposure only when the regime appears statistically supportive.

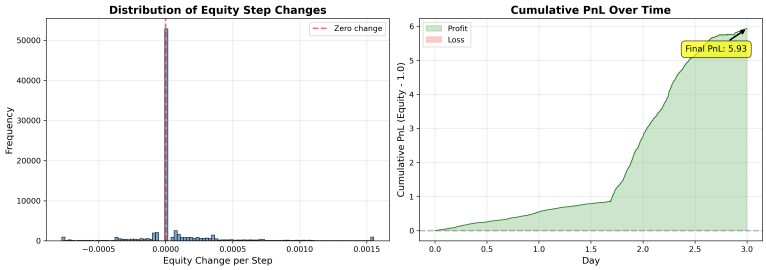

Figure 4: Left: Distribution of equity step changes during the test window (with a zero-change reference), showing that most decisions produce small realized impact with mild positive skew, consistent with a weak-but-persistent microstructure edge. Right: Cumulative PnL over time, illustrating how incremental gains compound into a materially positive final outcome.

4.3.4 Qualitative Analysis of Key Trading Periods

To understand agent behavior beyond aggregate metrics, we examined specific high- and low-performance windows within the test set. The strongest sustained performance occurred at steps 48,373 to 53,373 (approximately 1.4 hours), where equity increased from $1.85\times$ to $2.85\times$ (+54.1% return), corresponding to a momentum regime where cross-asset lead-lag signals were reliable. Conversely, the weakest 5,000-step window (steps 77,420 to 82,420) produced only +0.72% growth at higher equity levels ($6.7\times$), which suggests regime degradation and thus increased conservatism from Kelly sizing. The maximum drawdown over the 3-day test period was only -1.64%, lasting approximately 6 seconds.

Training loss dynamics supported our early-stopping decision: the volatility agent's TD loss decreased from 68.9 to 36.4 over epochs 1–3 before increasing in epoch 6, indicating overfitting. The ϵ-greedy exploration decayed to 0.01 within 50,000 steps, meaning the agent spent over 99% of training in exploitation—a potential limitation on strategy exploration.

4.3.5 Strategy Selection as a Learned Regime Map

The learned strategy mixture in Figure 5 provides an interpretable sanity check against our feature-screening prior. On the spot side, the agent allocates meaningful mass to cross-asset and flow-based hypotheses, consistent with the lead-lag prior (Section 2.2). On the volatility side, the agent favors persistence and mean-reversion hypotheses, aligning with the strong volatility autocorrelations in Table 3.

This reinforces the central claim of the paper: the agent is not discovering microstructure from scratch, but learning *when* each hypothesis is valid, and doing so at the appropriate timescale. The hierarchical decision frequencies appear to function as an implicit regularizer by preventing the volatility controller from reacting to noise at directional timescales.

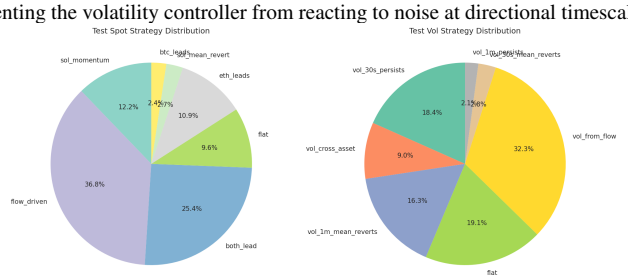

Figure 5: **Test-time strategy usage under the learned hierarchical policy.** Left: Spot strategy selection frequencies, showing a heavy allocation to cross-asset and flow-based hypotheses with a non-trivial abstention mass, consistent with short-horizon lead-lag priors. Right: Volatility strategy selection frequencies, emphasizing persistence/mean-reversion and flow-linked regimes at longer horizons. These distributions serve as an interpretability check that the agent learns *when* to deploy each hypothesis rather than relying on a single fixed predictor.

4.3.6 Financial Interpretation and Practical Caveats

Although the test results are directionally encouraging, they should be interpreted as a proof of concept rather than a production trading claim. The current evaluation omits transaction costs, slippage, and market impact, all of which are first-order at sub-one-second decision frequencies and eat away at our $\leq 10^{-4}$ average gains. In particular, even modest taker-fee assumptions could materially reduce the effective edge suggested by the step-level distribution.

Additionally, Kelly-style sizing, even when fractional and volatility-blended, can overstate deployable leverage under real-world constraints (latency, partial fills, and adverse selection). A more realistic live deployment would likely require explicit cost-aware rewards, conservative turnover penalties, and stress-testing across longer and more diverse regimes.

Overall, the results support the feasibility of hierarchical RL with online adaptive microstructure models: the agent learns an interpretable regime-selection policy whose out-of-sample behavior is consistent with cross-asset lead-lag intuition for returns and short-horizon volatility clustering for risk control. If we want to be "bullish" on the impact of our results, we have effectively created an "upper bound" on RL-based performance in the perp-trading game!

5 Discussion and Future Work

5.1 Summary of Contributions

We presented a hierarchical reinforcement learning framework for cryptocurrency trading that integrates domain-informed strategy libraries with data-driven regime selection across two timescales. The agents learn *when* to deploy interpretable hypotheses whose parameters adapt online via EWMA regression (Section 4.1.2).

At the macro level, our daily SOL spot agent leverages cross-asset momentum signals from SOL, BTC, ETH, and S&P 500, demonstrating that even coarse-grained, discrete-action RL can benefit from multi-asset context beyond a buy-and-hold prior. At the micro level, our design factorizes the problem into directional and volatility controllers with asynchronous decision frequencies (1s vs. 30–60s), coupled through a fractional, volatility-blended Kelly position-sizing layer. This modular decomposition improves interpretability and enables targeted debugging of failure modes (strategy calibration vs. regime selection vs. sizing).

Empirically, our micro-level test performance is positive out of sample (final equity $6.93\times$, Sharpe ≈ 0.98), with small but consistent per-step edges that compound over many decisions. The learned strategy mixtures provide an additional sanity check: spot selections place meaningful weight on cross-asset lead and flow-based hypotheses, while volatility selections emphasize persistence/mean-reversion and flow-linked risk regimes.

5.2 Key Insights and Takeaways

Learning regime selection is a strong bias in low signal-to-noise ratio markets. This constraint produces stable out-of-sample compounding despite extreme in-sample training variability. Asynchronous control improves robustness. Separating controllers at different frequencies (Section 4.1.1) reduces action-churn and dampens over-reaction to noise. Cross-asset information dominates short-horizon SOL direction, with the agent frequently preferring BTC/ETH-led hypotheses (see Section 2.2). Sizing is the main amplifier of both edge and overfitting. The risk controls in Section 4.1.5 are necessary to prevent training-era equity explosions from dominating checkpoint selection.

5.3 Limitations and Caveats

Several limitations are important for interpretation. First, we do not model transaction costs, slippage, or market impact, which are first-order at sub-one-second cadences; even modest taker fees could materially reduce the effective edge or shift the optimal policy toward lower turnover. Second, the system optimizes a single traded asset (SOL), leaving diversification and cross-asset risk budgeting unexplored. Third, the agent does not observe its current position, which simplifies the learning problem but prevents inventory-aware execution and hedging. Finally, our evaluation window may not capture rare stress regimes, so longer multi-regime backtests are needed before any deployment claims.

5.4 Future Research Directions

The most immediate extension is transaction-cost-aware learning. Incorporating maker/taker fees and simple slippage models directly into the reward would turn the current framework into a more realistic turnover-vs-edge optimizer. A closely related improvement is adding execution-aware features (spread, order book imbalance, depth) and allowing the policy to choose order types, potentially via a dedicated execution sub-agent.

A second direction is multi-asset generalization. Extending the strategy-selection framework to jointly trade BTC, ETH, and SOL could improve risk-adjusted returns through diversification and enable explicit correlation-aware sizing at the portfolio level.

Third, position-aware state representations and modest inventory penalties could unlock more realistic risk control at high frequency. Fourth, regime-specialist ensembles may offer improved robustness, where a meta-controller routes decisions to agents trained for distinct volatility/trend environments.

Fourth, exploring alternative RL algorithms (e.g. actor-critic or offline RL) and richer uncertainty quantification could further improve stability in low-signal settings and reduce reliance on early-epoch checkpointing.

Finally, an especially natural extension of our micro-level results is moving from directional perp trading into explicit volatility trading. Because our strongest signals appear to concentrate in short-horizon volatility persistence/reversion (e.g. the dominance of recent realized volatility over 30s to 5m scales), a more realistic next step is to express these forecasts through options structures such as straddles/strangles, or through delta-hedged volatility strategies where feasible. This would let the system "trade the uncertainty" directly rather than treating volatility solely as an input to position sizing.

Respository

Code for data processing, feature engineering, training environments, and both macro- and micro-level agents is available at: https://github.com/ethanfarah/solana-perps-cs230.

Contributions

Ethan Farah

Ethan led the design and implementation of the high-frequency framework, including the hierarchical spot/volatility decomposition, online EWMA regression strategy family, dual DQN regime selectors with asynchronous decision frequencies, and the calibrated fractional Kelly position-sizing pipeline with volatility blending.

Ethan built the micro-level data pipeline from Binance `aggTrades`, resampling to a shared 100ms grid and engineering the 420-dimensional cross-asset feature set spanning returns, EWMA momentum, flow imbalance, volume-weighted flow imbalance, realized volatility, and volume intensity with explicit lag structure. He implemented forward return and volatility target construction aligned to the hierarchical decision horizons.

Ethan performed the large-scale lead-lag correlation analysis that informed the final hypothesis library and helped justify the hierarchical timescale separation.

Ethan implemented chronological train/validation/test protocols, checkpoint selection, and state-restoration safeguards to prevent evaluation leakage. He also produced the micro-level training diagnostics, strategy distribution analyses, and test performance summaries.

Ethan drafted the majority of the micro-level methods, results, and financial-microstructure framing, and integrated findings into the overall narrative.

Ethan ran the microstructure training on an AWS EC2 `r7a.4xlarge` instance (132 GB RAM) and implemented memory-aware data engineering (chunked parquet workflows and post-target `float32` casting), enabling scalable training despite only being able to hold roughly a 20-day slice of the micro-level dataset in RAM at once.

Daniel Yang

Daniel led the development of the daily SOL trading framework, including cross-asset momentum feature construction from SOL, BTC, ETH, and S&P 500 data, the discrete action formulation (buy/hold/sell), and the macro-level DQN training and evaluation pipeline.

Daniel automated the acquisition and alignment of Binance kline data and S&P 500 price series, and constructed the macro-level feature dataset with consistent timestamp handling and train/validation/test splits.

Daniel implemented and evaluated buy-and-hold and random/bogo-style baselines for comparison, and generated macro-level performance metrics for inclusion in the final paper (pending final statistics for the abstract).

Daniel contributed to the dataset description, macro-level methods, and results editing, and provided iterative feedback on system design and experimental framing.

Joint Contributions

Both Ethan and Daniel collaborated on the project vision, experimental design, literature review, and integration of macro- and micro-level narratives into a single hierarchical RL story. Both authors contributed to iteration on paper structure, figure selection, and final proofreading.

References

[1] Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.

[2] Kelly, J.L. (1956). A new interpretation of information rate. *Bell System Technical Journal*, 35(4), 917–926.

[3] Ackerer, D., Hugonnier, J., & Jermann, U. (2024). Perpetual Futures Pricing. Working paper.

[4] Moody, J., Wu, L., Liao, Y., & Saffell, M. (1998). Reinforcement Learning for Trading. *Advances in Neural Information Processing Systems*, 11.

[5] Liu, X.-Y., Yang, H., Gao, J., & Wang, C. D. (2021). FinRL: Deep Reinforcement Learning Framework to Automate Trading in Quantitative Finance. *ACM International Conference on AI in Finance (ICAIF)*.

[6] Gort, B. J. D., Liu, X.-Y., Sun, X., Gao, J., Chen, S., & Wang, C. D. (2022). Deep Reinforcement Learning for Cryptocurrency Trading: Practical Approach to Address Backtest Overfitting. arXiv:2209.05559.

[7] CoinMarketCap. (2025). Bitcoin (BTC) price, market cap, and statistics. Accessed December 2025.

[8] CoinMarketCap. (2025). Ethereum (ETH) price, market cap, and statistics. Accessed December 2025.

[9] CoinMarketCap. (2025). Solana (SOL) price, market cap, and statistics. Accessed December 2025.

[10] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.

[11] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., & Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362.

[12] McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference (SciPy)*, 51–56.

Appendix A: Financial Background/Intuition

Momentum vs. Mean Reversion

Markets often exhibit two opposing behaviors that can dominate at different times. Momentum captures the idea that trends persist: if BTC has been rising over the past few seconds, it may continue rising in the immediate future because large orders are frequently split into smaller executions, news diffusion is not instantaneous, and trend-following behavior can reinforce short-term direction.

On the other hand, mean reversion is the idea that prices bounce back after moving too far from a local fair value: if SOL deviates strongly from a BTC- or ETH-implied reference, arbitrage and correlation-based trading can revert the gap as temporary order-flow imbalances are absorbed and overreactions are corrected. The core difficulty is that these regimes can switch unpredictably. Momentum works until it does not, and mean reversion works until a genuine trend begins. Our RL agent is designed to adaptively select between these competing hypotheses as market conditions evolve.

Order Flow Imbalance

At any moment, markets consist of buyers and sellers, and order flow imbalance provides a simple proxy for directional pressure. We define imbalance as $\text{Imbalance} = \frac{\text{Buy Volume} - \text{Sell Volume}}{\text{Buy Volume} + \text{Sell Volume}}$. An imbalance close to $+1$ indicates buyer-dominated flow and strong upward pressure, a value near 0 reflects a balanced market, and a value close to -1 indicates seller-dominated pressure. When imbalance is sustained, it can predict short-term continuation. For instance, if BTC shows persistent buy pressure, related assets such as SOL may follow as cross-asset participants adjust positions.

Volatility Prediction

Volatility, which we treat as the standard deviation of returns, measures the magnitude and uncertainty of price movement. Concretely, for log returns $r_t = \log\left(\frac{p_t}{p_{t-1}}\right)$, we compute volatility over a window of length w as $\sigma_{t,w} = \sqrt{\frac{1}{w-1}\sum_{i=0}^{w-1}\left(r_{t-i} - \bar{r}_{t,w}\right)^2}$ where $\bar{r}_{t,w}$ is the mean return over the same window. High volatility can increase potential profit per trade but also increases uncertainty, which typically calls for smaller position sizes. Volatility also functions as a regime indicator because it is known to cluster, meaning high-volatility periods tend to persist. For our system, predicting volatility is therefore not only a risk-management tool but also a way to adapt strategy selection and sizing to the prevailing market regime.